

Noser Engineering Ops Agent

Incident-Triage auf Autopilot

NOSER ENGINEERING

Betrieb kostet Entwicklerzeit

- Sentry-Alarm per E-Mail
- Manuelle Recherche nötig
- 20–40 Min. bis zur Diagnose
- Jeder Incident ein Neustart
- Low-prio Bugs gehen verloren

20–40

Minuten bis zur Diagnose

pro Incident – auch wenn der Fehler trivial ist. Sentry-Mail lesen, Logs suchen, Pipeline prüfen, Code öffnen: alles von Hand.

Zeit, die in der Feature-Entwicklung fehlt.

DIE LÖSUNG

Vom Alarm direkt zur Diagnose

VORHER

- **Sentry-Alarm per E-Mail**
  Entwickler wird aus dem Flow gerissen
- **Manuelle Recherche nötig**
  Logs, ADO-Pipelines, Codebase – alles einzeln
- **20–40 Min. bis zur Diagnose**
  Pro Incident, egal wie trivial
- **Kein strukturiertes Gedächtnis**
  Nächster Incident = Neustart von vorn
- **Low-prio Bugs gehen verloren**
  Zu unwichtig für sofort, zu wichtig zum Löschen

→

NACHHER

- **Sentry-Webhook trifft Agent direkt**
  Kein manueller Eingriff nötig
- **Automatisches Scoring**
  Urgent / Nice-to-know / Irrelevant
- **Agent holt Kontext selbst**
  Logs, Sourcecode, ADO-Pipeline in Sekunden
- **Teams-Post mit Diagnose**
  Befund + Fixvorschlag strukturiert als Card
- **Monday Digest für Low-prio**
  Nichts geht verloren, nichts unterbricht

NOSER ENGINEERING

Integration macht den Unterschied

Sentry

Webhook · Neue Issues & Regressions

Microsoft Teams

ChatOps · @-Mention im Channel

Support-Mailbox

Microsoft Graph · workid-support@noser.com

OPS AGENT

GPT-6-Luna / GPT-6.1-Sol

17 Tools verfügbar

Sentry · ADO · Azure

Log Analytics · Twilio · Graph

① Triage & Score

② Kontext holen

③ Antwort senden

Ops Cockpit im Browser

Urgent-Post in Teams

Diagnose + Fixvorschlag als Teams Card

Monday Digest

Wöchentliche Zusammenfassung (Low-prios)

Draft PR (on demand)

OpenCode – nur auf expliziten Wunsch

NOSER ENGINEERING

Drei Integrationsmuster

PUSH

Webhooks & Requests

Systeme melden sich beim Agent.

`POST /webhooks/sentry`
Sentry · HMAC-signiert

`POST /api/messages`
Teams · Bot Framework (JWT)

`GET /ui/ · /api/ui/*`
Ops Cockpit · Entra ID

PULL

Tools

Der Agent holt sich Kontext selbst.

Sentry · Azure DevOps · Log Analytics
Azure ARM · Twilio · Microsoft Graph

REST- und SDK-Calls mit Token oder Managed Identity

ausgelöst vom LLM per Function Calling

SCHEDULE

Jobs

Zeitgesteuert oder auf Abruf.

Monday Digest · Mo 08:00
Graph-Subscription · stündlich
Memory-Outcomes · stündlich
Codefix-Worker · on demand

Azure Container App Jobs: gleiches Image, anderer Entrypoint

Stack — **Node 22 · TypeScript** · **Fastify** · **Zod** · **Bot Framework SDK** · **Azure OpenAI** · **Cosmos DB** · **Container Apps** · **Terraform**

Drei Integrationsmuster

PUSH

Webhooks & Requests

Systeme melden sich be

`POST /webhooks/sentry`
Sentry · HMAC-signiert

`POST /api/messages`
Teams · Bot Framework (JW

`GET /ui/ · /api/ui/*`
Ops Cockpit · Entra ID

PULL

Tools

Identity

ausgelöst vom LLM per Function Calling

SCHEDULE

Jobs

auf Abruf.

Mo 08:00

· stündlich

· stündlich

Codefix-Worker · on demand

Azure Container App Jobs: gleiches Image, anderer Entrypoint

Was gibt es da für Risiken?

Stack: Node 22 · TypeScript | Fastify | Zod | Bot Framework SDK | Azure OpenAI | Cosmos DB | Container Apps | Terraform

Webhook: Security

1. **Sentry ruft `POST /webhooks/sentry` auf**

   bei neuen Issues und Regressions – JSON-Payload plus Signatur-Header

2. **Signatur prüfen – sonst 401**

   HMAC-SHA256 über Raw Body vs. `sentry-hook-signature`

3. **Normalisieren zu `IncidentSignal`**

   Titel, Level, Environment, betroffene User, Häufigkeit pro Stunde, Release-Bezug

4. **Idempotent speichern**

   Schlüssel `event:sentry:<id>` – doppelte Zustellung wird erkannt

5. **`202 Accepted` – sofort**

   Triage läuft entkoppelt weiter – Sentry wartet nie auf das LLM

src/handlers/hmac.ts

```ts
const expected = createHmac("sha256", secret)
  .update(rawBody)        // exakte Wire-Bytes
  .digest("hex");

if (expected.length !== signature.length)
  return false;           // → 401
return timingSafeEqual(
  Buffer.from(expected),
  Buffer.from(signature));
```

NOSER ENGINEERING

Erst Regeln, dann LLM, dann Guardrails

1 · REGELN

1 Pre-Score aus YAML

Level fatal / error	**80 / 40**
Release < 24 h	**+20**
≥ 100 User betroffen	**+30**
≥ 50× pro Stunde	**+20**
Bürozeiten (CH)	**+5**
Cap Dev/Test · Staging	**45 · 65**

Summe gedeckelt auf 0–100

→

2 · LLM

2 Agent mit Tools

- Holt Evidenz über Tools (≤ 8 Schritte)
- Antwortet als Structured Output: JSON Schema, strict
- `classification · confidence · reasoning · actionPlan · suggestedFix`

→

3 · CODE

3 Guardrails

- **Safety-Override:** fatal oder Score ≥ 80 ⇒ immer urgent
- **Env-Policy:** urgent auf Dev/Test ⇒ nice-to-know
- **Ausnahme:** Prod-, Kunden- oder Security-Signal

URGENT

Teams-Card mit Diagnose & Fix

NICE-TO-KNOW

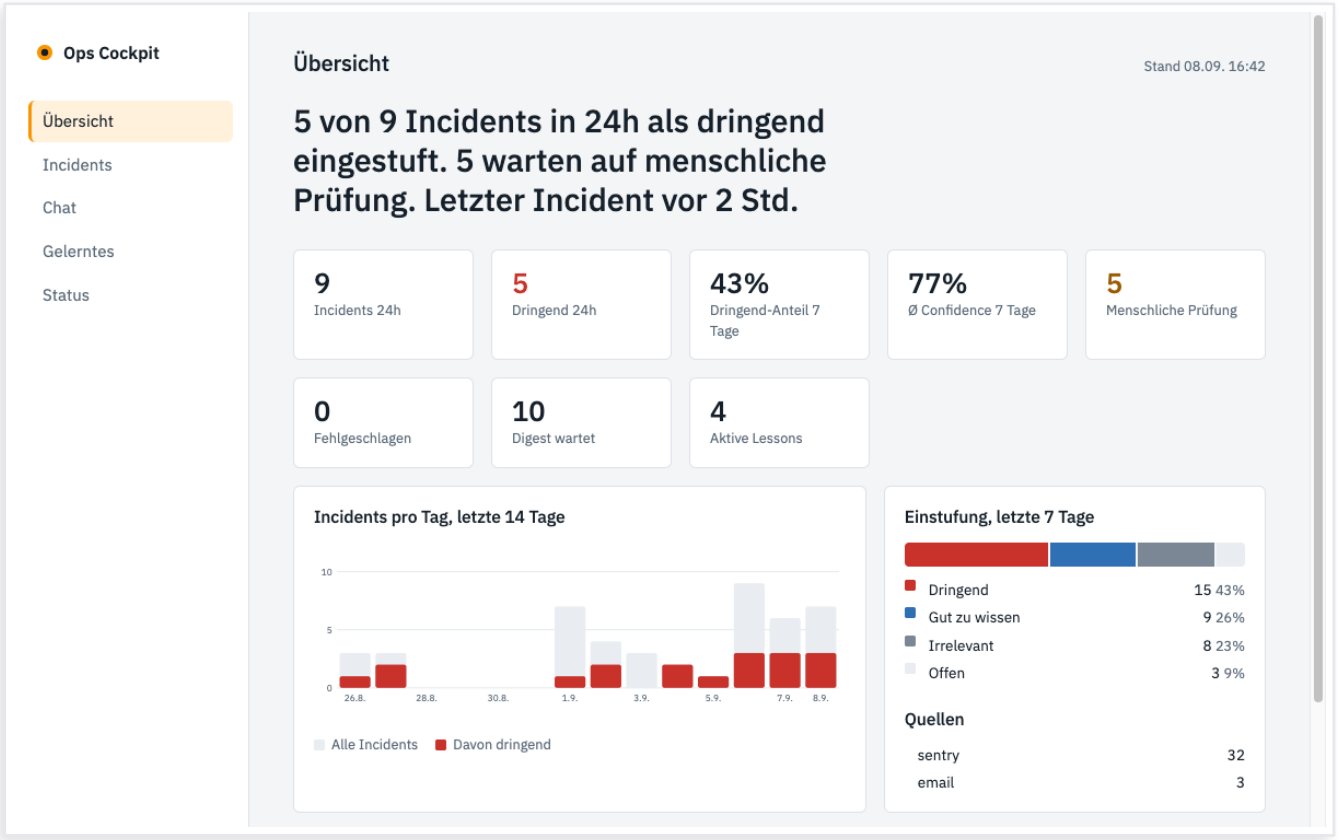
Digest-Queue → Monday Digest

IRRELEVANT

Nur gespeichert – kein Lärm im Channel

NOSER ENGINEERING

Das Ops Cockpit: der Agent im Browser



Übersicht mit KPIs, 14-Tage-Verlauf und Einstufung · Demo-Modus mit synthetischen Daten

5 Views

Übersicht · Incidents mit Re-Triage · Chat · Gelerntes · Status

Vanilla-SPA, kein Build

HTML, CSS, ES-Module – ausgeliefert vom Fastify-Prozess unter `/ui/`

API im selben Server

`/api/ui/*` nutzt Repository, Memory und `runChatOps` – gleiche Tools wie Teams

Entra ID

MSAL.js (Auth Code + PKCE), Backend prüft JWT und Rolle `Cockpit.User`

NOSER ENGINEERING

ZUSAMMENGEFASST

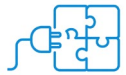
Agentic Aufbau: vier Bausteine

LLM als Orchestrator



- GPT-5.4 für Triage, GPT-5.4-mini für Chat & Digest
- Agent-Loop: Triage ≤ 8, Chat ≤ 16 Schritte
- Structured Output: JSON Schema statt Freitext

Tools für Anbindungen



- Zod-Schema = Vertrag zwischen LLM und API
- 17 Tools über 6 Schnittstellen, lesend als Default
- Schreiben nur explizit: Draft-PR auf Wunsch

Evaluation für Qualitätssicherung



- Golden Set + Korrekturfälle aus dem Betrieb
- pnpm eval:triage misst die Pass-Rate
- ~8.7k Zeilen Tests, externe APIs gemockt (msw)

Guardrails by Design



- HMAC, Bot-JWT und Entra ID an jedem Eingang
- Safety-Override & Env-Policy im Code, nicht im Prompt
- Probes nur auf Allowlist-Domains, PRs nur als Draft

NOSER ENGINEERING

PROJEKT IN ZAHLEN

Was wir gelernt haben

4
Wochen bis zum ersten Wurf

~11k
Zeilen Code (TS + UI)

~8.7k
Zeilen Tests

17
Tools · 6 Schnittstellen



Integration > Modell

Der Mehrwert entsteht durch Kontext aus echten Systemen. Ohne Sentry, Logs und Code ist auch das beste LLM nur ein guter Rater.



Regeln + LLM + Code

Deterministisch, was bekannt ist. LLM, was offen ist. Guardrails im Code – nicht als Bitte im Prompt.



Vertrauen bauen

Draft statt Merge, Vorschlag statt Aktion, jede Entscheidung begründet. Korrekturen fliessen als Lessons zurück.

NOSER ENGINEERING

Danke für die Aufmerksamkeit

PS: Der grösste Blocker war die User Acceptance



Tobias Ritscher
AI Solutions Engineer
Noser Engineering AG

076 515 75 58
tobias.ritscher@noser.com

NOSER ENGINEERING